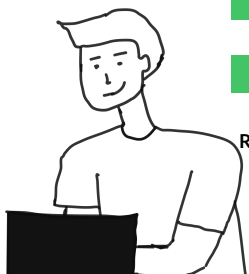
Integer Programming

Lecture 7

ETF2480 & ETF5248 - S2, 2026
Jessica Leung, PhD, CAP-X, SFHEA

Today's Goal

- Integer programming
- Modelling techniques
- Algorithms
- Guidelines for strong formulations
- Traveling salesman problem



Reading:

Bertsimas and Tsitsiklis, *Introduction to Linear Optimization*, Chapters 10 and 11.

Winston, *Operations Research*, Chapter 9.

Vanderbei, *Linear Programming*, Chapter 23.

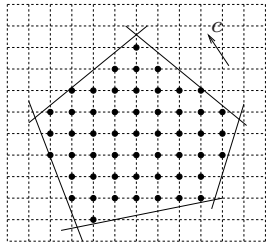
Integer Programming

In **discrete** optimisation problems, we seek to find a solution $x^\star$ in a discrete set $\mathcal{P}$ that optimises (minimises or maximises) an objective function $f_0(x)$ defined for all $x \in \mathcal{P}$

- discrete optimisation problems arise in a great variety of contexts
- a natural and systematic way to study a broad class of discrete optimisation problems is to express them as integer programming problems
- the (linear) integer programming problem is the same as the linear programming problem except that some of the variables are restricted to take integer values

Integer linear program (IP) or (ILP):

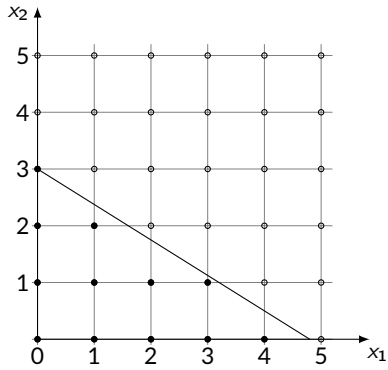
$$\begin{array}{ll} \text{maximize} & c^T x \\ \text{subject to} & Ax \leq b \\ & x \in \mathbb{Z}^n \end{array}$$



- **mixed integer program (MIP):** if only some of the variables are integer
- **0-1 (Boolean) linear program:** variables are restricted to be either 0 or 1

Example 1: 2-variable IP

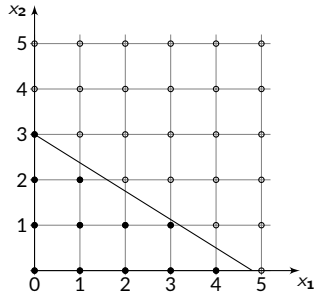
$$
\begin{aligned}
\text{maximize} \quad & 3x_1 + 4x_2 \\
\text{subject to} \quad & 5x_1 + 8x_2 \leq 24 \\
& x_1, x_2 \geq 0 \\
& x_1, x_2 \in \mathbb{Z}
\end{aligned}
$$



- what is the integer solution?
- what is the relaxed (LP) solution?
- can LP be used to solve the IP?

Example 1: 2-variable IP

$$
\begin{aligned}
\text{maximize} \quad & 3x_1 + 4x_2 \\
\text{subject to} \quad & 5x_1 + 8x_2 \leq 24 \\
& x_1, x_2 \geq 0 \\
& x_1, x_2 \in \mathbb{Z}
\end{aligned}
$$



how to solve?

- solve LP (ignore integrality), get $x_{\text{lp}}^\star = (4.8, 0)$ and $p(x_{\text{lp}}^\star) = 14.4$
- round: get $x_{\text{r}} = (5, 0) \rightarrow$ infeasible!
- truncate: get $x_{\text{t}} = (4, 0) \rightarrow p = 12$
- optimal: $x_{\text{ip}}^\star = (3, 1) \rightarrow p(x_{\text{ip}}^\star) = 13$

both rounding and truncating do not work, need a different method!

Example 2: work-scheduling

A post office requires different number of full-time employees on different days of the week (see Table). Union rules state that each employee must work five consecutive days and then receive two days off. For example, an employee who works Monday to Friday must be off on Saturday and Sunday. The post office wants to meet its daily requirements using only full-time employees.

Formulate an LP that the post office can use to minimise the number of full-time employees who must be hired.

Table: Requirements for post office

Day	Mon	Tue	Wed	Thu	Fri	Sat	Sun
# of employees	17	13	15	19	14	16	11

Example 2: solution

Let x_i be the number of employees beginning work on day i, then

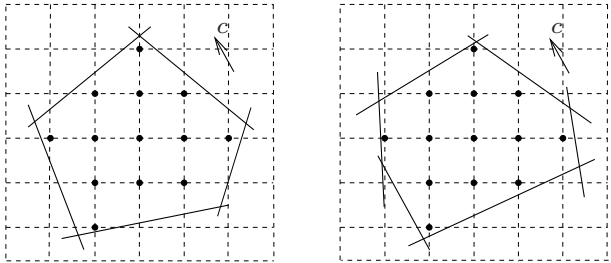
$$\begin{aligned}
\text{minimize} \quad & \sum_{i=1}^{7} x_i \\
\text{subject to} \quad & x_1 + x_4 + x_5 + x_6 + x_7 \geq 17 \\
& x_1 + x_2 + x_5 + x_6 + x_7 \geq 13 \\
& x_1 + x_2 + x_3 + x_6 + x_7 \geq 15 \\
& x_1 + x_2 + x_3 + x_4 + x_7 \geq 19 \\
& x_1 + x_2 + x_3 + x_4 + x_5 \geq 14 \\
& x_2 + x_3 + x_4 + x_5 + x_6 \geq 16 \\
& x_3 + x_4 + x_5 + x_6 + x_7 \geq 11 \\
& x_i \geq 0 \quad i = 1, \ldots, 7
\end{aligned}$$

- **LP solution:** $x_{\text{lp}} = (1\frac{1}{3}, 3\frac{1}{3}, 2, 7\frac{1}{3}, 0, 3\frac{1}{3}, 5) \rightarrow p(x_{\text{lp}}) = 22\frac{1}{3}$
- **rounded solution:** $x_{\text{r}} = (2, 4, 2, 8, 0, 4, 5) \rightarrow p(x_{\text{r}}) = 25$
- **truncated solution:** $x_{\text{t}} = (1, 3, 2, 7, 0, 3, 5) \rightarrow$ infeasible!
- **integer solution:** $x_{\text{ip}} = (4, 4, 2, 6, 0, 4, 3) \rightarrow p(x_{\text{ip}}) = 23$

Linear programming relaxation

relaxation: remove the constraints $x \in \mathbf{Z}^n$

- provides an upper bound on the optimal value of the integer LP
- if solution of relaxation is integer, then it solves the integer LP



equivalent IP formulations can have different LP relaxations

Modelling techniques

- binary choice
- forcing constraints
- disjunctive (either-or) constraints
- set-covering problems
- if-then constraints

Binary choice

An important use of a binary variable x is to encode a choice between two alternatives: we may set x to zero or one, depending on the chosen alternative.

Knapsack problem: we are given n items, each with a weight (w_j) and a value (v_j). Given a bound W on the weight that can be carried in a knapsack, we would like to select items to maximise the total value.

In order to model this problem, we define a binary variable x_j which is 1 if item j is chosen, and 0 otherwise. The problem can then be formulated as follows:

$$\begin{aligned} \text{maximize} \quad & \sum_{j=1}^{n} v_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} w_j x_j \leq W \\ & x_j \in \{0,1\}, \quad j = 1, \ldots, n. \end{aligned}$$

Example 3

Stockco is considering 6 investments. Each investment yields a determined NPV and requires a certain cash flow at the present time (see table). Currently Stockco has $14,000 available for investment. What is the optimal strategy?

investment	1	2	3	4	5	6
cash required (1000s)	$5	$7	$4	$3	$4	$6
NPV added (1000s)	$16	$22	$12	$8	$11	$19

Forcing constraints

A very common feature in discrete optimisation problems is that certain decisions are dependent.

More specifically, suppose decision A can be made only if decision B has also been made.

In order to model such a situation, we introduce binary variables x (respectively, y) equal to 1 if decision A (respectively, B) is chosen, and 0 otherwise

The dependence of the two decisions can be modelled using the constraint:

$$x \leq y$$

Example 4: facility location

Suppose we are given n potential facility locations and a list of m clients who need to be serviced from these locations. There is a fixed cost c_j of opening a facility at location j, while there is a cost d_{ij} of serving client i from facility j. The goal is to select a set of facility locations and assign each client to one facility, while minimising the total cost.

Solution: let $y_j = 1$ if facility j is selected, and 0 otherwise. In addition, let $x_{ij} = 1$ if client i is served by facility j, and 0 otherwise. Then, the problem can be formulated as follows:

$$\begin{aligned}
\text{minimize} \quad & \sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m} \sum_{j=1}^{n} d_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad \forall i, \\
& x_{ij} \leq y_j, \quad \forall i, j, \\
& x_{ij}, y_j \in \{0, 1\}, \quad \forall i, j.
\end{aligned}$$

Constraint $x_{ij} \leq y_j$ captures the fact that if there is no facility at location j ($y_j = 0$), then client i cannot be served there, and we must have $x_{ij} = 0$.

Relations between variables

at most one of the binary variables x_j can be one:

$$\sum_{j=1}^{n} x_j \leq 1$$

exactly one of the binary variables x_j should be one:

$$\sum_{j=1}^{n} x_j = 1$$

at least two of the binary variables x_j should be one:

$$\sum_{j=1}^{n} x_j \geq 2$$

exactly three of the binary variables x_j should be one:

$$\sum_{j=1}^{n} x_j = 3$$

Restricted range

Suppose we want to restrict variable x to take values in a set $\{a_1, a_2, \ldots, a_m\}$.

We can achieve this by introducing m binary variables y_j, $j = 1, \ldots, m$, and the constraints

$$x = \sum_{j=1}^{m} a_j y_j, \quad \sum_{j=1}^{m} y_j = 1, \quad y_j \in \{0, 1\}.$$

Example: suppose x can only take values $0.0, 0.5, 1.0, 1.5, 2.0$. We can model this with the following constraints:

$$\begin{aligned} x &= 0.0y_1 + 0.5y_2 + 1.0y_3 + 1.5y_4 + 2.0y_5 \\ y_1 &+ y_2 + y_3 + y_4 + y_5 = 1 \\ y_j &\in \{0, 1\} \quad j = 1, \ldots, 5. \end{aligned}$$

Disjunctive constraints

Let x be a nonnegative decision vector. Suppose that we are given two constraints

$$a^T x \geq b \quad \text{and} \quad c^T x \geq d,$$

in which $a \geq 0$ and $c \geq 0$.
We would like to model a requirement that at least one of the constraints is satisfied. In order to achieve this, we define a binary variable y and impose the constraints:

$$\begin{aligned} a^T x &\geq yb, \\ c^T x &\geq (1-y)d, \\ y &\in \{0,1\}. \end{aligned}$$

More generally, suppose we are given m constraints

$$a_i^T x \geq b_i, \quad i = 1, \ldots, m,$$

where $a_i \geq 0$ for each i, and require that at least k of them are satisfied.
We can achieve this by introducing m binary variables y_i, $i = 1, \ldots, m$, and the constraints:

$$a_i^T x \geq y_i b_i, \quad \sum_{i=1}^{m} y_i \geq k, \quad y_i \in \{0,1\}.$$

Generalizing disjunctive constraints I

Suppose we are given two constraints of the form

$$f(x) \leq 0$$
$$g(x) \leq 0$$

We want to make sure that at least one of them is satisfied.

In order to achieve this, we define a binary variable y and impose the constraints

$$f(x) \leq My$$
$$g(x) \leq M(1-y)$$

where M is a number chosen large enough to ensure that

$$f(x) \leq M \quad \text{and} \quad g(x) \leq M$$

are satisfied for all values of x that satisfy the other constraints in the problem.
This type of constraint is sometimes called 'Either-or constraints' in some textbooks.

Generalizing disjunctive constraints II

Suppose we are given m constraints of the form

$$f_i(x) \leq 0 \qquad i = 1, \ldots, m$$

We want to make sure that at least k of them is satisfied. That means at most $m - k$ of them can be violated.

In order to achieve this, we define m binary variable $y_i, i = 1, \ldots, m$ and impose the constraints

$$f_i(x) \leq M y_i$$
$$\sum_i y_i \leq m - k$$

where M is a number chosen large enough.

Example

Dorian Auto is considering manufacturing three types of autos: compact, midsize and large. The resources required for, and profits yielded by, each type of car are shown in the table. Currently, 6,000 tons of steel and 60,000 hours of labor are available. For production of a type of a car to be economically feasible, at least 1,000 cars of that type must be produced. Formulate an IP to maximize Dorian's profit.

	car type		
resource	compact	midsize	large
Steel required	1.5 tons	3 tons	5 tons
Labor required	30 hours	25 hours	40 hours
Profit yielded ($)	2,000	3,000	4,000

Suppose x_1 is the number of compact cars produced. Then, the economical feasibility constraint for compact cars is

$$x_1 \leq 0 \quad \text{or} \quad x_1 \geq 1000$$

Define $f(x) = x_1$ and $g(x) = 1000 - x_1$, add the following set of constraints:

$$x_1 \leq M_1 y_1$$
$$1000 - x_1 \leq M_1(1 - y_1)$$
$$y_1 \in \{0, 1\}$$

If-then constraints

In many applications, we want to ensure that if a constraint $f(x) > 0$ is satisfied, then the constraint $g(x) \geq 0$ must be satisfied, while if $f(x) > 0$ is not satisfied, then $g(x) \geq 0$ may or may not be satisfied.

In other words, we want to ensure that $f(x) > 0$ implies $g(x) \geq 0$.

To ensure this, we include the following constraints in the formulation

$$\begin{aligned} -g(x) &\leq My \\ f(x) &\leq M(1-y) \\ y &\in \{0,1\} \end{aligned}$$

Set-covering problem

In a **set-covering problem**, each member of a given set (call it set $\mathcal{A}$) must be "covered" by an acceptable member of some other set (call it set $\mathcal{B}$).

The **objective** of a set-covering problem is to minimize the number of elements in set $\mathcal{B}$ that are required to cover all the elements in set $\mathcal{A}$.

Example: There are six cities in Kilroy County (cities 1–6). The county must determine where to build fire stations. The county wants to build the minimum number of fire stations needed to ensure that at least one fire station is within 15 minutes (driving time) of each city. The times (in minutes) required to drive between the cities in Kilroy County are shown in the table below. Formulate an IP that will tell Kilroy how many fire stations should be built and where they should be located.

	To					
From	City 1	City 2	City 3	City 4	City 5	City 6
City 1	0	10	20	30	30	20
City 2	10	0	25	35	20	10
City 3	20	25	0	15	30	20
City 4	30	35	15	0	15	25
City 5	30	20	30	15	0	14
City 6	20	10	20	25	14	0

Set-covering problem

Formulation: let x_i equal to 1 if a fire station is built in city i and 0 otherwise. Then the objective is

$$\text{minimize} \quad \sum_{i=1}^{6} x_i$$

The constraints are a little more complicated. The table below indicates which locations can reach the city in 15 minutes or less.

City	Within 15 minutes
City 1	1, 2
City 2	1, 2, 6
City 3	3, 4
City 4	3, 4, 5
City 5	4, 5, 6
City 6	2, 5, 6

To ensure that at least one fire station is within 15 minutes of city 1, we add the constraint

$$x_1 + x_2 \geq 1$$

In a similar fashion we add constraints for the remaining cities.

Set-covering problem

The full formulation of Kilroy County problem is

$$
\begin{aligned}
\text{minimize} \quad & \sum_{i=1}^{6} x_i \\
\text{subject to} \quad & x_1 + x_2 \geq 1 \\
& x_1 + x_2 + x_6 \geq 1 \\
& x_3 + x_4 \geq 1 \\
& x_3 + x_4 + x_5 \geq 1 \\
& x_4 + x_5 + x_6 \geq 1 \\
& x_2 + x_5 + x_6 \geq 1 \\
& x_i \in \{0, 1\} \quad i = 1, \ldots, 6
\end{aligned}
$$

solution: $x^\star = (0, 1, 0, 1, 0, 0)$ with $p^\star = 2$

Thus, Kilroy County can build two fire stations: one in city 2 and one in city 4.

note: here set $\mathcal{A}$ is the cities of Kilroy County and set $\mathcal{B}$ is the set of fire stations.

Live Coding Time!

Let’s code the set covering problem together!

Workshop: Beyond Efficiency - Equity Considerations

While minimizing the number of fire stations is financially efficient, it ignores the positive economic impacts of municipal infrastructure. Building a fire station creates local jobs, improves neighborhood infrastructure, and increases community safety.
Historically, some cities in Kilroy County have been underinvested in. To account for this, the county assigns an **Equity Impact Score (EIS)** to each city (from 1 to 10), where a higher score represents a historically marginalized community that would benefit most from local investment.

City	Equity Impact Score (E_i)
City 1	8
City 2	2
City 3	4
City 4	3
City 5	9
City 6	7

Goal: Ensure 15-minute coverage for all cities while balancing the *cost* of building stations with the *equity impact* of where they are built.

Multi-Objective Formulation: Linear Combination

We now have two competing objectives:

Minimize total stations (Efficiency): $f_1(x) = \sum_{i=1}^{6} x_i$

Maximize equity impact (DEI): $f_2(x) = \sum_{i=1}^{6} E_i x_i$

To solve this in an IP framework, we can combine these into a single objective function using a **linear combination**. We assign weights w_1 and w_2 to represent the relative importance of each goal. Because we want to minimize stations but maximize equity, we subtract the equity term:

$$\text{minimize} \quad Z = w_1 \sum_{i=1}^{6} x_i - w_2 \sum_{i=1}^{6} E_i x_i$$

The full objective function becomes:

$$\begin{aligned} \text{minimize} \quad & w_1(x_1 + x_2 + x_3 + x_4 + x_5 + x_6) \\ & - w_2(8x_1 + 2x_2 + 4x_3 + 3x_4 + 9x_5 + 7x_6) \end{aligned}$$

The constraints remain exactly the same, ensuring every city is covered within 15 minutes!

Analyzing the Trade-off

By adjusting the weights w_1 and w_2, policymakers can explore the **Pareto frontier** of possible solutions.

Scenario A: Pure Efficiency ($w_1 = 1, w_2 = 0$)

Solution: $x^\star = (0, 1, 0, 1, 0, 0)$

Cost: 2 stations (Built in City 2 and City 4)

Total Equity Impact: $2 + 3 = 5$

Scenario B: Balanced Approach ($w_1 = 10, w_2 = 1$)

Solution: $x^\star = (1, 0, 1, 0, 1, 0)$

Cost: 3 stations (Built in City 1, City 3, and City 5)

Total Equity Impact: $8 + 4 + 9 = 21$

Workshop Discussion: In Scenario A, the stations are built in areas with low equity scores. By investing in just **one additional station** (Scenario B), the county more than *quadruples* the equity impact (from 5 to 21) while still meeting all response-time constraints. Which plan should the county choose, and why?

Algorithms

Unlike LPs, IPs are very difficult to solve.
In fact, no **efficient** general algorithm is known for their solution.
There are three main categories of algorithms:

- **Exact algorithms**: guaranteed to find an optimal solution, but may take an exponential number of iterations (e.g. branch-and-bound, cutting plane)
- **Approximation algorithms**: provide in polynomial time a suboptimal solution with a bound on suboptimality.
- **Heuristic algorithms**: provide a suboptimal solution, but without a guarantee on its quality.

Branch-and-bound algorithm

Consider an integer program

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{subject to} \quad & x \in \mathcal{P} \end{aligned}$$

where $\mathcal{P}$ is a finite set
general idea:

- recursively partition $\mathcal{P}$ into smaller sets $\mathcal{P}_i$ and solve subproblems

$$\begin{aligned} \text{maximize} \quad & c^T x \\ \text{subject to} \quad & x \in \mathcal{P}_i \end{aligned}$$

- use LP relaxations to discard subproblems that don't lead to a solution

Branch-and-bound algorithm: overview

- Solve the LP relaxation of the original IP. If the solution is integer, then stop.
- If not, pick a variable whose value in the LP relaxation is fractional. For the sake of argument, suppose that this variable is x_1 and its value in the LP relaxation is 3.75. We can then exclude this value by imposing the restrictions $x_1 \leq 3.0$ and $x_1 \geq 4.0$.
- If the original IP is denoted P_0, then we might denote these two new IPs by P_1, where $x_1 \leq 3.0$ is imposed, and P_2, where $x_1 \geq 4.0$ is imposed. The variable x_1 is then called a **branching variable**, and we are said to have branched on x_1.
- We now apply the same idea to these two IPs, solving the corresponding LP relaxations and, if necessary, selecting branching variables. In so doing we generate what is called a **search tree**. The IPs generated by the search procedure are called the **nodes** of the tree, with P_0 designated as the **root node**. The leaves of the tree are all the nodes from which we have not yet branched.

In general, if we reach a point at which we can solve or otherwise dispose of all leaf nodes, then we will have solved the original IP.

Branch-and-bound algorithm: fathomed and incumbent nodes

The best integer solution found at any point in the search is called the **incumbent**. At the start of the search, we have no incumbent.

Suppose that we have just solved the LP relaxation of some node in the search tree. If all variables have integer values, then we know we have found a feasible solution to the original IP.

There are **two** important steps that we then take:

- We designate this node as **fathomed**. It is not necessary to branch on this node; it is a permanent leaf of the search tree.
- If the integer feasible solution that we have just found has a better objective function value than the current incumbent (or if we have no incumbent), then we record this solution as the new incumbent, along with its objective function value.

There are two other possibilities that can lead to a node being fathomed.

- the LP relaxation at this node is infeasible.
- the value of the LP relaxation is smaller than that of the current incumbent.

In both cases, it is not necessary to branch on this node.

Branch-and-bound algorithm: best bound and gap

First, observe that the **objective value for the incumbent** is a **valid lower bound** on the optimal solution of the given IP (assuming the original IP is a maximization problem).

Second, at any time during the branch-and-bound search we also have a **valid upper bound**, sometimes called the **best bound**. This bound is obtained by taking the maximum of the optimal objective values of all of the current leaf nodes.

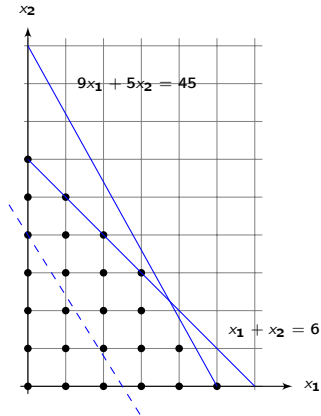
The difference between the current upper and lower bounds is called **optimality gap** (or the **gap**). It can be relative (% of the LP objective value) and absolute.

When the optimality gap is zero we have demonstrated optimality.

Branch-and-bound algorithm: example

Consider the following integer program and its feasible region:

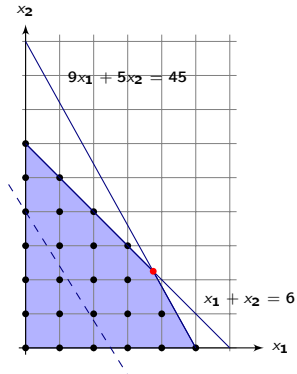
$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1, x_2 \in \mathbb{Z}
\end{aligned}
$$



Branch-and-bound algorithm: example

Start with LP relaxation (subproblem P_0):

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0
\end{aligned}
$$



upper bound = ∞, **lower bound** = $-\infty$
solution to subprobem P_0: $x_1 = 3.75$, $x_2 = 2.25$, $p^\star_{rl} = p^\star_0 = 41.25$
new **upper bound** = 41.25

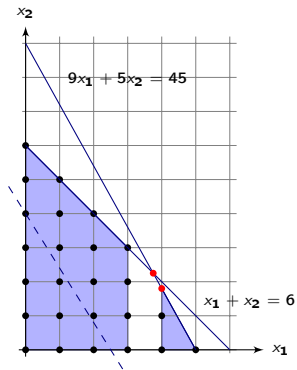
Branch-and-bound algorithm: example

branch on x_1 and create two subproblems:

- subproblem P_1: P_0 and $x_1 \geq 4$
- subproblem P_2: P_0 and $x_1 \leq 3$

subproblem P_1:

$$\begin{array}{rl} \text{maximize} & 8x_1 + 5x_2 \\ \text{subject to} & x_1 + x_2 \leq 6 \\ & 9x_1 + 5x_2 \leq 45 \\ & x_1, x_2 \geq 0 \\ & x_1 \geq 4 \end{array}$$



upper bound = 41.25, **lower bound** = $-\infty$
solution to P_1: $x_1 = 4$, $x_2 = 1.8$, $p_1^\star = 41$

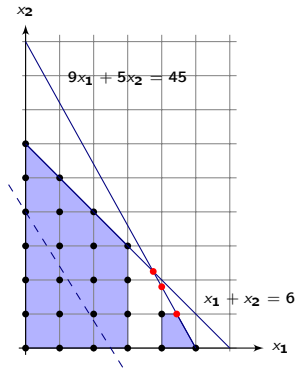
Branch-and-bound algorithm: example

branch subproblem P_1 on x_2 and create two subproblems:

- P_3: P_1 and $x_2 \geq 2$ (**infeasible**, fathomed)
- P_4: P_1 and $x_2 \leq 1$

subproblem P_4:

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1 \geq 4 \\
& x_2 \leq 1
\end{aligned}
$$



upper bound = 41.25, **lower bound** = $-\infty$
solution to P_4: $x_1 = 4.44$, $x_2 = 1$, $p_4^\star = 40.55$

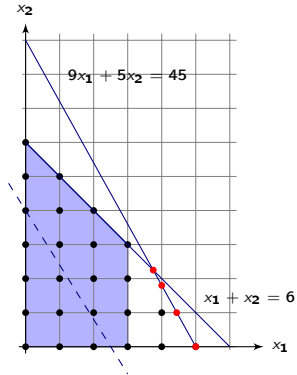
Branch-and-bound algorithm: example

branch subproblem P_4 on x_1 and create two subproblems:

- P_5: P_4 and $x_1 \geq 5$
- P_6: P_4 and $x_1 \leq 4$

subproblem P_5:

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1 \geq 4 \\
& x_2 \leq 1 \\
& x_1 \geq 5
\end{aligned}
$$



upper bound = 41.25, **lower bound** = $-\infty$
solution to P_5: $x_1 = 5$, $x_2 = 0$, $p_5^\star = 40$ (solution is integer, hence incumbent)
new **lower bound** = $p_5^\star$ = 40

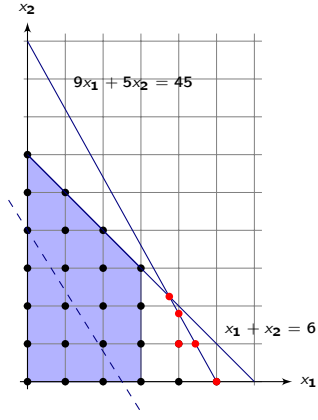
Branch-and-bound algorithm: example

branch subproblem P_4 on x_1 and create two subproblems:

- P_5: P_4 and $x_1 \geq 5$
- P_6: P_4 and $x_1 \leq 4$

subproblem P_6:

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1 \geq 4 \\
& x_2 \leq 1 \\
& x_1 \leq 4
\end{aligned}
$$



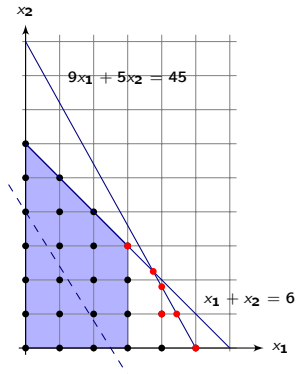
upper bound = 41.25, **lower bound** = 40
solution to P_6: $x_1 = 4$, $x_2 = 1$, $p_6^\star = 37$ (lower than **lower bound**, fathomed)

Branch-and-bound algorithm: example

back to subproblem P_2:

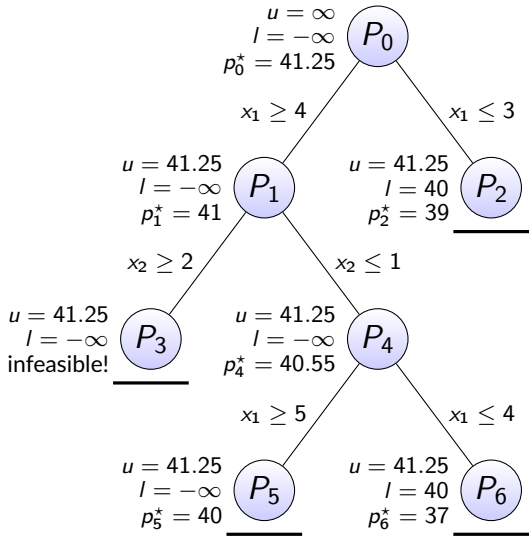
- P_2: P_0 and $x_1 \leq 3$

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1 \leq 3
\end{aligned}
$$



upper bound = 41.25, **lower bound** = 40
solution to P_2: $x_1 = 3$, $x_2 = 3$, $p_2^\star = 39$ (lower than **lower bound**, fathomed)

Branch-n-bound tree



Branch-and-bound algorithm: solution

- no more unsolved subproblems
- subproblem P_5 gives the best integer solution (current incumbent)
- hence, solution to the IP is: $x_1^\star = 5$, $x_2^\star = 0$, $p^\star = 40$

Remarks:

- there are many different approaches to determine which subproblems should be solved next
- the one we used is called **backtracking**
- another commonly used approach is **jumptracking**: solve all the problems created by the most recent branching, then branch on the node with the best objective value
- when solving a difficult IP, we can be happy with a near-optimal solution, so an algorithm can be terminated when it reaches certain optimality gap, e.g., 5% or 1%

Cutting plane algorithm

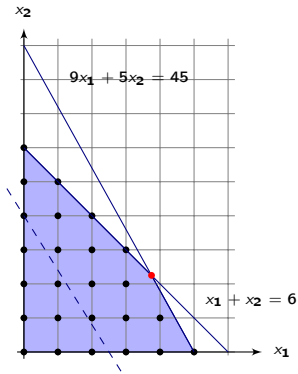
A generic cutting plane algorithm:

1. Solve the LP relaxation. Let $x^\star$ be an optimal solution.
2. If $x^\star$ is integer, stop; $x^\star$ is an optimal solution to the IP.
3. If not, add a linear inequality constraint to the LP relaxation that all integer solutions to IP satisfy, but $x^\star$ does not; go to Step 1.

Cutting plane algorithm: example

Consider the IP from previous example:

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1, x_2 \in \mathbb{Z}
\end{aligned}
$$



solution to LP relaxation: $x_1 = 3.75$, $x_2 = 2.25$, $p^\star_{rl} = 41.25$ (**upper bound**)

Cutting plane algorithm: example

Table: optimal tableau for the LP relaxation

z	x_1	x_2	s_1	s_2	**rhs**
1	0	0	1.25	0.75	41.25
0	0	1	2.25	-0.25	2.25
0	1	0	-1.25	0.25	3.75

- choose any constraint in the LP relaxation's optimal tableau in which a basic variable is fractional (say, constraint 2)

$$x_1 - 1.25s_1 + 0.25s_2 = 3.75$$

- define $\lfloor a \rfloor = \max\{m \in \mathbb{Z} \mid m \leq a\}$ (**floor**), then any a can be written as $\lfloor a \rfloor + f$ where $0 \leq f < 1$
- rewrite chosen constraint as $x_1 - 2s_1 + 0s_2 - 3 = 0.75 - 0.75s_1 - 0.25s_2$
- add the following constraint to the LP relaxation's optimal tableau: fractional part of the chosen constraint ≤ 0

$$0.75 - 0.75s_1 - 0.25s_2 \leq 0$$

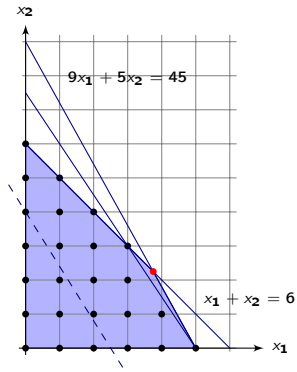
- recall that $s_1 = 6 - x_1 - x_2$ and $s_2 = 45 - 9x_1 - 5x_2$ hence the cut is

$$3x_1 + 2x_2 \leq 15$$

Cutting plane algorithm: example

resulting LP relaxation with the first cutting plane:

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& 3x_1 + 2x_2 \leq 15
\end{aligned}
$$



properties of the cut:

- any feasible point of the IP satisfies the cut
- the current solution to the LP relaxation does not satisfy the cut

Cutting plane algorithm: remarks

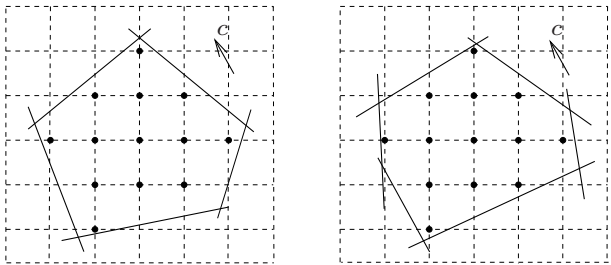
- the performance of a cutting plane method depends critically on the choice of cuts
- we used a cut proposed by Ralph E. Gomory in 1958: the first finitely terminating algorithm for integer programming
- a difficulty with general purpose cutting plane algorithms is that the added inequalities cut only a very small piece of the feasible set of the LP relaxation and as a result the performance of such algorithms has not been impressive
- modern solvers use many different cuts simultaneously

Linear programming relaxation

relaxation: remove the constraints $x \in \mathbb{Z}^n$

provides an upper bound on the optimal value of the integer LP

if solution of relaxation is integer, then it solves the integer LP



equivalent IP formulations can have different LP relaxations

Integer programming models are sensitive to formulations

in LP, a good formulation is one that has a small number of variables (n) and constraints (m), because the computational complexity of the problem grows polynomially in n and m

in addition, given the availability of efficient algorithms for LPs, the choice of a formulation, although important, does not critically affect our ability to solve the problem

the situation in integer programming is drastically different: extensive computational experience suggests that the choice of a formulation is crucial

Guidelines for strong formulations

Recall the facility location problem (left) and consider its **aggregate** formulation (right):

$$\begin{aligned}
\text{minimize} \quad & \sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m} \sum_{j=1}^{n} d_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad \forall i, \\
& x_{ij}, y_j \in \{0,1\}, \quad \forall i, j, \\
& x_{ij} \leq y_j, \quad \forall i, j.
\end{aligned}$$

$$\begin{aligned}
\text{minimize} \quad & \sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m} \sum_{j=1}^{n} d_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad \forall i, \\
& x_{ij}, y_j \in \{0,1\}, \quad \forall i, j, \\
& \sum_{i=1}^{m} x_{ij} \leq m y_j, \quad \forall j.
\end{aligned}$$

constraint $\sum_{i=1}^{m} x_{ij} \leq m y_j$ is equivalent to the constraints $x_{ij} \leq y_j$, $i = 1, \ldots, m$, hence the feasible set and the optimal cost is the same for both formulations

however, the aggregate formulation has $m + n$ constraints, while the original formulation has $m + mn$ constraints

Guidelines for strong formulations

In order to compare the two formulations, consider their corresponding LP relaxations:

$$
\begin{aligned}
\text{minimize} \quad & \sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m} \sum_{j=1}^{n} d_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad \forall i, \\
& x_{ij} \leq y_j, \quad \forall i, j, \\
& 0 \leq x_{ij} \leq 1, \quad \forall i, j, \\
& 0 \leq y_j \leq 1, \quad \forall j.
\end{aligned}
$$

$$
\begin{aligned}
\text{minimize} \quad & \sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m} \sum_{j=1}^{n} d_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j=1}^{n} x_{ij} = 1, \quad \forall i, \\
& \sum_{i=1}^{m} x_{ij} \leq m y_j, \quad \forall j, \\
& 0 \leq x_{ij} \leq 1, \quad \forall i, j, \\
& 0 \leq y_j \leq 1, \quad \forall j.
\end{aligned}
$$

Guidelines for strong formulations

- clearly, the feasible set of the LP relaxation of the original formulation (left) is closer to the set of integer solutions than the LP relaxation of the aggregate formulation (right)
- if we let p_{ip} be the optimal cost of the IP problem, and let p_{lp} and p_{alp} be the optimal costs of the two LP relaxations we have introduced, then the following inequality holds:

$$p_{\text{alp}} \leq p_{\text{lp}} \leq p_{\text{ip}}$$

- many methods for solving IP problems depend on the availability of lower bounds such as p_{lp}
- thus, the original formulation is preferable (stronger) than the aggregate formulation, despite the fact that the latter has significantly fewer constraints

Quality of IP formulation

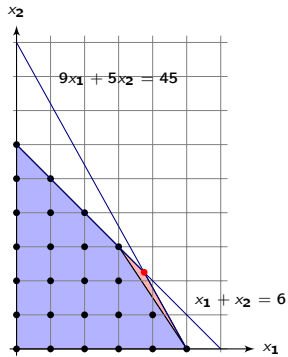
The **quality of a formulation** of an IP with feasible set $\mathcal{T}$, can be judged by the closeness of the feasible set of its LP relaxation to the convex hull of $\mathcal{T}$.

In particular, consider two formulations A and B of the same IP. If we denote by $\mathcal{P}_A$ and $\mathcal{P}_B$ the feasible sets of the corresponding LP relaxations, we consider formulation A to be at least as **strong** as formulation B if

$$\mathcal{P}_A \subset \mathcal{P}_B$$

Example: feasible set, convex hull, and feasible set of LP relaxation

$$
\begin{aligned}
\text{maximize} \quad & 8x_1 + 5x_2 \\
\text{subject to} \quad & x_1 + x_2 \leq 6 \\
& 9x_1 + 5x_2 \leq 45 \\
& x_1, x_2 \geq 0 \\
& x_1, x_2 \in \mathbb{Z}
\end{aligned}
$$



- feasible set $\mathcal{T}$ is all black circles
- convex hull of $\mathcal{T}$ is the blue shaded area
- feasible set of LP relaxation is the blue+red shaded area

Traveling salesman problem (TSP)

Given an undirected graph $G = (N, E)$ with n nodes, and costs c_{ij} for every edge $(i, j) \in E$, the goal is to find a *tour* (a cycle that visits all nodes) of minimum cost.

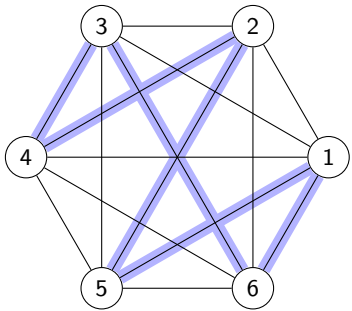
In order to model the problem, we define for every edge $(i, j) \in E$ a variable x_{ij}, which is 1 if edge (i, j) is included in the tour, and zero, otherwise. This leads to the following formulation:

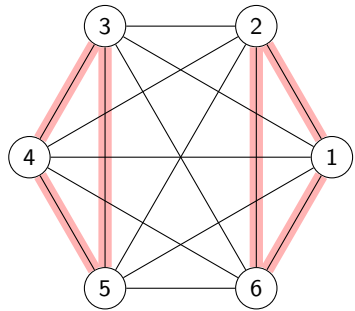
$$
\begin{aligned}
\text{minimize} \quad & \sum_{(i,j) \in E} c_{ij} x_{ij} \\
\text{subject to} \quad & \sum_{j:(i,j) \in E} x_{ij} = 1 \quad \text{for all } i \in N \\
& \sum_{i:(i,j) \in E} x_{ij} = 1 \quad \text{for all } j \in N \\
& x_{ij} \in \{0, 1\} \quad \text{for all } (i, j) \in E
\end{aligned}
$$

The issue with this formulation is that it can produce subtours.

Example

- a proper tour $1 - 5 - 2 - 4 - 3 - 6 - 1$ (left) and
- two subtours $1 - 2 - 6 - 1$ and $3 - 4 - 5 - 3$ (right)
- both are feasible solutions for TSP on previous slide





Need to introduce **subtour elimination constraints** to avoid solutions with subtours.

Subtour elimination constraints

The Miller, Tucker and Zemlin (MTZ, 1960) formulation:

$$u_i - u_j + n x_{ij} \leq n - 1 \quad \text{for all } (i,j) \in E$$
$$u_i \geq 0 \text{ and integer} \quad \text{for all } i \in N$$

The Dantzig, Fulkerson and Johnson (DFJ, 1954) formulation:

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad \text{for all } S \subset N, |S| \geq 2$$

Both formulations result in subtour elimination and correctly solve the TSP problem.

However, the formulations are vastly different:

- TMZ has a much smaller number of constraints:

$$|E| + n \quad \text{vs.} \quad \binom{n}{2} + \binom{n}{3} + \cdots + \binom{n}{n-1}$$

- TMZ introduces n new integer variables, DFJ does not
- DFJ is much stronger

Summary on integer programming

- integer programming is a powerful modelling framework that provides great flexibility
- the price for this flexibility is that integer programming seems to be a much more difficult problem than linear programming
- unlike linear programming problems, strong formulations are central to being able to solve integer programming problems efficiently
- the quality of a formulation is judged by the closeness of its linear programming relaxation to the convex hull of integer feasible solutions
- however, strong formulations occasionally require a very large number of constraints
- unfortunately, all existing algorithms have a disadvantage: either they take an exponential amount of time in the worst case or they only give an approximate solution

Workshop: Modified Giapetto - Integer Programming Setup

Context: Giapetto released new models! The profit and labor requirements have changed slightly. We want to maximize profit, but we *cannot* sell fractional trains or soldiers.

Integer Programming Formulation:

$$\begin{aligned} \text{maximize} \quad & Z = 4x_1 + 3x_2 \\ \text{subject to} \quad & 3x_1 + 2x_2 \leq 100 \quad \text{(finishing)} \\ & 2x_1 + 3x_2 \leq 102 \quad \text{(carpentry)} \\ & x_1, x_2 \geq 0 \text{ and } \textbf{integer} \end{aligned}$$

We run the continuous LP Relaxation (ignoring the integer constraint) and get the following **optimal LP tableau**:

BV	Z	x_1	x_2	s_1	s_2	**RHS**
Z	1	0	0	$6/5$	$1/5$	$702/5$ $(= 140.4)$
x_1	0	1	0	$3/5$	$-2/5$	$96/5$ $(= 19.2)$
x_2	0	0	1	$-2/5$	$3/5$	$106/5$ $(= 21.2)$

Workshop Questions:

- Why is this LP solution invalid for the IP problem?
- Write the equation for the x_1 row and derive the **Gomory Fractional Cut**.
- Substitute the slack variables (s_1, s_2) to express this cut in terms of our original variables, x_1 and x_2. What does the cut look like?